Data Structures Lab Programs

Questions with complete Python solutions

Contents

Group A: Arrays, Searching and Sorting (3 questions)

Group B: Stacks, Queues and Linked List (4 questions)

Group C: Hashing (3 questions)

Group D: Graphs and Trees (6 questions)

Each program has been run and tested with Python 3. Menu-driven programs show a menu in the terminal; run each one with `python filename.py`.

Group A: Arrays, Searching and Sorting

Question 1

Write a Python program to manage the borrowing records of books in a library. Implement the following functionalities:

- Compute the average number of books borrowed by all library members.
- Find the book with the highest and lowest number of borrowings in the library.
- Count the number of members who have not borrowed any books (denoted by a borrow count of 0).
- Display the most frequently borrowed book (i.e., the mode of borrow counts).

After performing, determine the time and Space complexity of each operation.

Python code (A1.py)

```
# Library borrowing records
members = {"Aarav": 5, "Bela": 0, "Chirag": 3, "Divya": 5,
           "Esha": 0, "Farhan": 2, "Gita": 3, "Hari": 5}          # member -> books borrowed
books = {"Python Basics": 12, "Data Structures": 25, "Algorithms": 18,
         "Operating Systems": 4, "DBMS": 25, "Networks": 4}        # book -> times borrowed


def average_borrowed(member_counts):
    """Average books borrowed per member."""
    total = 0
    for c in member_counts:          # O(n)
        total += c
    return total / len(member_counts) if member_counts else 0


def highest_lowest_books(book_counts):
    """Return (highest_books, lowest_books); ties are all reported."""
    hi = lo = None
    for count in book_counts.values():       # one pass: O(m)
        hi = count if hi is None or count > hi else hi
        lo = count if lo is None or count < lo else lo
    highest = [b for b, c in book_counts.items() if c == hi]
    lowest = [b for b, c in book_counts.items() if c == lo]
    return (highest, hi), (lowest, lo)


def members_with_zero(member_counts):
    """Count members whose borrow count is 0."""
    zero = 0
    for c in member_counts:                  # O(n)
        if c == 0:
            zero += 1
    return zero


def mode_of_counts(counts):
    """Mode = the borrow-count value that occurs most often."""
    freq = {}
    for c in counts:                         # O(n) using a hash map
        freq[c] = freq.get(c, 0) + 1
    best = max(freq.values())
    return [v for v, f in freq.items() if f == best], best


if __name__ == "__main__":
    member_counts = list(members.values())

    print("Average books borrowed per member:", round(average_borrowed(member_counts), 2))

    (hb, hc), (lb, lc) = highest_lowest_books(books)
    print(f"Most borrowed book(s) : {hb} ({hc} times)")
    print(f"Least borrowed book(s): {lb} ({lc} times)")

    print("Members who never borrowed:", members_with_zero(member_counts))

    modes, freq = mode_of_counts(member_counts)
```

```
    print(f"Mode of member borrow counts: {modes} (occurs {freq} times)")
    book_modes, bfreq = mode_of_counts(list(books.values()))
    print(f"Mode of book borrow counts  : {book_modes} (occurs {bfreq} times)")

    print("""
Complexity (n = members, m = books)
Operation               Time        Space
Average                 O(n)        O(1)
Highest / lowest book   O(m)        O(1)  (O(m) if all ties are listed)
Members with zero       O(n)        O(1)
Mode (hash map)         O(n)        O(k)  k = distinct borrow counts <= n
""")
```

Question 2

In an e-commerce system, customer account IDs are stored in a list, and you are tasked with writing a program that implements the following:

- Linear Search: Check if a particular customer account ID exists in the list.
- Binary Search: Implement Binary search to find if a customer account ID exists, improving the search efficiency over the basic linear search.

Python code (A2.py)

```
# Linear search and binary search on customer account IDs

def linear_search(ids, target):
    """Check every element one by one. Time O(n), Space O(1)."""
    for i in range(len(ids)):
        if ids[i] == target:
            return i
    return -1


def binary_search(ids, target):
    """ids MUST be sorted. Time O(log n), Space O(1)."""
    low, high = 0, len(ids) - 1
    while low <= high:
        mid = (low + high) // 2
        if ids[mid] == target:
            return mid
        elif ids[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1


if __name__ == "__main__":
    ids = [10452, 10231, 10987, 10110, 10765, 10333, 10876, 10543]
    print("Customer IDs:", ids)

    target = int(input("Enter account ID to search: "))

    pos = linear_search(ids, target)
    print("Linear search :", f"found at index {pos}" if pos != -1 else "not found")

    sorted_ids = sorted(ids)             # binary search needs sorted data, O(n log n)
    pos = binary_search(sorted_ids, target)
    print("Binary search :", f"found at index {pos} of sorted list" if pos != -1 else "not found")

    print("""
            Best    Average    Worst     Space
Linear      O(1)    O(n)       O(n)      O(1)
Binary      O(1)    O(log n)   O(log n)  O(1)   (list must be sorted first)
""")
```

Question 3

In a company, employee salaries are stored in a list as floating-point numbers. Write a Python program that sorts the employee salaries in ascending order using the following two algorithms:

- Selection Sort: Sort the salaries using the selection sort algorithm.
- Bubble Sort: Sort the salaries using the bubble sort algorithm.

After sorting the salaries, the program should display top five highest salaries in the company.

Python code (A3.py)

```
# Sort employee salaries using Selection Sort and Bubble Sort

def selection_sort(arr):
    """Time O(n^2) in all cases, Space O(1)."""
    a = arr[:]
    n = len(a)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        a[i], a[min_idx] = a[min_idx], a[i]
    return a


def bubble_sort(arr):
    """Time O(n^2) worst/average, O(n) best (already sorted), Space O(1)."""
    a = arr[:]
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:          # no swaps -> already sorted
            break
    return a


def top_five(sorted_salaries):
    """Salaries are in ascending order, so the last five are the highest."""
    return sorted_salaries[-5:][::-1]


if __name__ == "__main__":
    salaries = [45000.50, 72000.00, 38000.75, 91000.25, 56000.00,
                64000.80, 83000.10, 29000.00, 99000.99, 51000.45]
    print("Original salaries:", salaries)

    sel = selection_sort(salaries)
    print("\nSelection Sort (ascending):", sel)

    bub = bubble_sort(salaries)
    print("Bubble Sort    (ascending):", bub)

    print("\nTop five highest salaries:")
    for rank, s in enumerate(top_five(sel), start=1):
        print(f"  {rank}. {s:.2f}")
```

Group B: Stacks, Queues and Linked List

Question 1

Implementing a real-time undo/redo system for a text editing application using a Stack data structure. The system should support the following operations:

- Make a Change: A new change to the document is made.
- Undo Action: Revert the most recent change and store it for potential redo.
- Redo Action: Reapply the most recently undone action.
- Display Document State: Show the current state of the document after undoing or redoing an action.

Python code (B1.py)

```
# Undo / Redo system for a text editor using two stacks

class TextEditor:
    def __init__(self):
        self.undo_stack = []      # changes currently applied (top = latest)
        self.redo_stack = []      # changes that were undone

    def make_change(self, text):
        self.undo_stack.append(text)
        self.redo_stack.clear()   # a new change invalidates the redo history
        print(f"Change made: '{text}'")

    def undo(self):
        if not self.undo_stack:
            print("Nothing to undo.")
            return
        change = self.undo_stack.pop()
        self.redo_stack.append(change)
        print(f"Undone: '{change}'")

    def redo(self):
        if not self.redo_stack:
            print("Nothing to redo.")
            return
        change = self.redo_stack.pop()
        self.undo_stack.append(change)
        print(f"Redone: '{change}'")

    def display(self):
        print("Document state:", " ".join(self.undo_stack) if self.undo_stack else "<empty>")


if __name__ == "__main__":
    editor = TextEditor()
    while True:
        print("\n1.Make Change  2.Undo  3.Redo  4.Display Document  5.Exit")
        ch = input("Choice: ").strip()
        if ch == "1":
            editor.make_change(input("Enter text to add: "))
        elif ch == "2":
            editor.undo()
        elif ch == "3":
            editor.redo()
        elif ch == "4":
            editor.display()
        elif ch == "5":
            break
        else:
            print("Invalid choice.")
# All operations (push / pop / peek) are O(1) time; space is O(n) for n changes.
```

Question 2

Implement a real-time event processing system using a Queue data structure. The system should support the following features:

- Add an Event: When a new event occurs, it should be added to the event queue.
- Process the Next Event: The system should process and remove the event that has been in the queue the longest.
- Display Pending Events: Show all the events currently waiting to be processed.
- Cancel an Event: An event can be canceled if it has not been processed.

Python code (B2.py)

```
# Real-time event processing system using a Queue (FIFO)
from collections import deque


class EventQueue:
    def __init__(self):
        self.queue = deque()
        self.next_id = 1

    def add_event(self, description):
        self.queue.append((self.next_id, description))      # enqueue at rear, O(1)
        print(f"Event #{self.next_id} added: {description}")
        self.next_id += 1

    def process_next(self):
        if not self.queue:
            print("No events to process.")
            return
        eid, desc = self.queue.popleft()                    # dequeue from front, O(1)
        print(f"Processed event #{eid}: {desc}")

    def display_pending(self):
        if not self.queue:
            print("No pending events.")
            return
        print("Pending events (oldest first):")
        for eid, desc in self.queue:
            print(f"  #{eid}: {desc}")

    def cancel_event(self, event_id):                       # O(n) search
        for item in self.queue:
            if item[0] == event_id:
                self.queue.remove(item)
                print(f"Event #{event_id} cancelled.")
                return
        print("Event not found (already processed or invalid ID).")


if __name__ == "__main__":
    eq = EventQueue()
    while True:
        print("\n1.Add Event  2.Process Next  3.Display Pending  4.Cancel Event  5.Exit")
        ch = input("Choice: ").strip()
        if ch == "1":
            eq.add_event(input("Event description: "))
        elif ch == "2":
            eq.process_next()
        elif ch == "3":
            eq.display_pending()
        elif ch == "4":
            eq.cancel_event(int(input("Event ID to cancel: ")))
        elif ch == "5":
            break
        else:
            print("Invalid choice.")
```

Question 3

A call center receives incoming calls, and each call is assigned a unique customer ID. The calls are answered in the order they are received. Your task is to simulate the call queue of a call center using a queue data structure.

- addCall(customerID, callTime): Add a call to the queue with the customer ID and the call time (in minutes).
- answerCall(): Answer and remove the first call from the queue.
- viewQueue(): View all calls currently in the queue without removing them.
- isQueueEmpty(): Check if the queue is empty.

Python code (B3.py)

```
# Call centre queue simulation
class Call:
    def __init__(self, customer_id, call_time):
        self.customer_id = customer_id
        self.call_time = call_time          # duration in minutes


class CallCenter:
    def __init__(self):
        self.queue = []                     # front = index 0

    def addCall(self, customerID, callTime):
        self.queue.append(Call(customerID, callTime))
        print(f"Call added: customer {customerID}, {callTime} min")

    def answerCall(self):
        if self.isQueueEmpty():
            print("No calls to answer.")
            return None
        call = self.queue.pop(0)            # first come, first served
        print(f"Answered call of customer {call.customer_id} ({call.call_time} min)")
        return call

    def viewQueue(self):
        if self.isQueueEmpty():
            print("Queue is empty.")
            return
        print("Calls in queue (front -> rear):")
        for pos, c in enumerate(self.queue, start=1):
            print(f"  {pos}. Customer ID: {c.customer_id}, Call time: {c.call_time} min")

    def isQueueEmpty(self):
        return len(self.queue) == 0


if __name__ == "__main__":
    cc = CallCenter()
    while True:
        print("\n1.Add Call  2.Answer Call  3.View Queue  4.Is Queue Empty?  5.Exit")
        ch = input("Choice: ").strip()
        if ch == "1":
            cid = input("Customer ID: ")
            t = int(input("Call time (minutes): "))
            cc.addCall(cid, t)
        elif ch == "2":
            cc.answerCall()
        elif ch == "3":
            cc.viewQueue()
        elif ch == "4":
            print("Empty" if cc.isQueueEmpty() else "Not empty")
        elif ch == "5":
            break
        else:
            print("Invalid choice.")
```

Question 4

Create a Student Record Management System using linked list

- Use a singly/doubly linked list to store student data (Roll No, Name, Marks).
- Perform operations: Add, Delete, Update, Search, and Sort.
- Display records in ascending/descending order based on marks or roll number.

Python code (B4.py)

```
# Student Record Management System using a doubly linked list
class Node:
    def __init__(self, roll, name, marks):
        self.roll, self.name, self.marks = roll, name, marks
        self.prev = self.next = None


class StudentList:
    def __init__(self):
        self.head = self.tail = None

    def search(self, roll):
        cur = self.head
        while cur:
            if cur.roll == roll:
                return cur
            cur = cur.next
        return None

    def add(self, roll, name, marks):
        if self.search(roll):
            print("Roll number already exists.")
            return
        node = Node(roll, name, marks)
        if not self.head:
            self.head = self.tail = node
        else:
            node.prev = self.tail
            self.tail.next = node
            self.tail = node
        print("Student added.")

    def delete(self, roll):
        node = self.search(roll)
        if not node:
            print("Student not found.")
            return
        if node.prev: node.prev.next = node.next
        else:         self.head = node.next
        if node.next: node.next.prev = node.prev
        else:         self.tail = node.prev
        print("Student deleted.")

    def update(self, roll, name, marks):
        node = self.search(roll)
        if not node:
            print("Student not found.")
            return
        node.name, node.marks = name, marks
        print("Record updated.")

    def sort(self, key):
        """Bubble sort (ascending) by 'roll' or 'marks' by swapping node data."""
        if not self.head:
            return
        swapped = True
        while swapped:
            swapped = False
            cur = self.head
            while cur.next:
                if getattr(cur, key) > getattr(cur.next, key):
                    cur.roll, cur.next.roll = cur.next.roll, cur.roll
                    cur.name, cur.next.name = cur.next.name, cur.name
                    cur.marks, cur.next.marks = cur.next.marks, cur.marks
                    swapped = True
                cur = cur.next
```

```
    def display(self, key="roll", descending=False):
        if not self.head:
            print("No records.")
            return
        self.sort(key)                      # list is now ascending by key
        print(f"{'Roll':<8}{'Name':<15}{'Marks':<8}")
        cur = self.tail if descending else self.head
        while cur:                          # backward walk uses 'prev' link
            print(f"{cur.roll:<8}{cur.name:<15}{cur.marks:<8}")
            cur = cur.prev if descending else cur.next


if __name__ == "__main__":
    sl = StudentList()
    while True:
        print("\n1.Add 2.Delete 3.Update 4.Search 5.Sort & Display 6.Exit")
        ch = input("Choice: ").strip()
        if ch == "1":
            sl.add(int(input("Roll No: ")), input("Name: "), float(input("Marks: ")))
        elif ch == "2":
            sl.delete(int(input("Roll No to delete: ")))
        elif ch == "3":
            sl.update(int(input("Roll No to update: ")), input("New name: "), float(input("New marks: ")))
        elif ch == "4":
            n = sl.search(int(input("Roll No to search: ")))
            print(f"Found: {n.roll}, {n.name}, {n.marks}" if n else "Not found.")
        elif ch == "5":
            key = "marks" if input("Sort by (r)oll / (m)arks: ").lower().startswith("m") else "roll"
            desc = input("Order (a)scending / (d)escending: ").lower().startswith("d")
            sl.display(key, desc)
        elif ch == "6":
            break
        else:
            print("Invalid choice.")
# add O(n) (duplicate check), delete/update/search O(n), sort O(n^2), display O(n)
```

Group C: Hashing

Question 1

Implement a hash table of size 10 and use the division method as a hash function. In case of a collision, use chaining. Implement the following operations:

- Insert(key): Insert key-value pairs into the hash table.
- Search(key): Search for the value associated with a given key.
- Delete(key): Delete a key-value pair from the hash table.

Python code (C1.py)

```
# Hash table of size 10: division method + separate chaining
class ChainedHashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]      # each slot holds a chain (list)

    def hash(self, key):
        return key % self.size                      # division method

    def insert(self, key, value):
        chain = self.table[self.hash(key)]
        for pair in chain:
            if pair[0] == key:                      # key exists -> update value
                pair[1] = value
                print(f"Key {key} updated.")
                return
        chain.append([key, value])                  # collision -> add to chain
        print(f"Inserted ({key}, {value}) at index {self.hash(key)}")

    def search(self, key):
        for k, v in self.table[self.hash(key)]:
            if k == key:
                return v
        return None

    def delete(self, key):
        chain = self.table[self.hash(key)]
        for pair in chain:
            if pair[0] == key:
                chain.remove(pair)
                print(f"Key {key} deleted.")
                return
        print("Key not found.")

    def display(self):
        for i, chain in enumerate(self.table):
            print(f"[{i}] -> " + (" -> ".join(f"({k}:{v})" for k, v in chain) or "empty"))


if __name__ == "__main__":
    ht = ChainedHashTable(10)
    while True:
        print("\n1.Insert  2.Search  3.Delete  4.Display  5.Exit")
        ch = input("Choice: ").strip()
        if ch == "1":
            ht.insert(int(input("Key (integer): ")), input("Value: "))
        elif ch == "2":
            v = ht.search(int(input("Key to search: ")))
            print(f"Value = {v}" if v is not None else "Key not found.")
        elif ch == "3":
            ht.delete(int(input("Key to delete: ")))
        elif ch == "4":
            ht.display()
        elif ch == "5":
            break
        else:
            print("Invalid choice.")
# Average O(1) per operation (load factor alpha = n/10), worst case O(n) if all keys collide.
```

Question 2

Design and implement a hash table of fixed size. Use the division method for the hash function and resolve collisions using linear probing. Allow the user to perform the following operations:

- Insert a key
- Search for a key
- Delete a key
- Display the table

Python code (C2.py)

```
# Fixed-size hash table: division method + linear probing
EMPTY, DELETED = None, "<deleted>"


class LinearProbingTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [EMPTY] * size

    def hash(self, key):
        return key % self.size

    def insert(self, key):
        if self.search(key) != -1:
            print("Key already present.")
            return
        idx, first_free = self.hash(key), -1
        for i in range(self.size):
            pos = (idx + i) % self.size             # linear probe: h, h+1, h+2 ...
            if self.table[pos] is EMPTY:
                if first_free == -1:
                    first_free = pos
                break
            if self.table[pos] == DELETED and first_free == -1:
                first_free = pos
        if first_free == -1:
            print("Hash table is full.")
            return
        self.table[first_free] = key
        print(f"Inserted {key} at index {first_free}")

    def search(self, key):
        idx = self.hash(key)
        for i in range(self.size):
            pos = (idx + i) % self.size
            if self.table[pos] is EMPTY:            # reached an empty slot -> absent
                return -1
            if self.table[pos] == key:
                return pos
        return -1

    def delete(self, key):
        pos = self.search(key)
        if pos == -1:
            print("Key not found.")
        else:
            self.table[pos] = DELETED               # tombstone keeps probe chains intact
            print(f"Key {key} deleted from index {pos}")

    def display(self):
        for i, v in enumerate(self.table):
            print(f"[{i}] {'-' if v is EMPTY else v}")


if __name__ == "__main__":
    ht = LinearProbingTable(10)
    while True:
        print("\n1.Insert  2.Search  3.Delete  4.Display  5.Exit")
        ch = input("Choice: ").strip()
        if ch == "1":
            ht.insert(int(input("Key: ")))
        elif ch == "2":
            p = ht.search(int(input("Key to search: ")))
```

```
            print(f"Found at index {p}" if p != -1 else "Not found.")
        elif ch == "3":
            ht.delete(int(input("Key to delete: ")))
        elif ch == "4":
            ht.display()
        elif ch == "5":
            break
        else:
            print("Invalid choice.")
```

Question 3

Implement a hash table with extendible hashing. The hash table should dynamically expand when the number of keys in a bucket exceeds a certain threshold. Perform the following operations:

- Insert(key): Insert key-value pairs into the hash table
- Search(key): Search for the value associated with a given key
- Delete(key): Delete a key-value pair from the hash table

Python code (C3.py)

```
# Extendible hashing: directory doubles / buckets split when a bucket overflows
class Bucket:
    def __init__(self, depth, capacity):
        self.depth = depth                  # local depth
        self.capacity = capacity
        self.items = {}


class ExtendibleHashTable:
    def __init__(self, bucket_capacity=2):
        self.capacity = bucket_capacity     # threshold for keys per bucket
        self.global_depth = 1
        self.directory = [Bucket(1, bucket_capacity), Bucket(1, bucket_capacity)]

    @staticmethod
    def _hash(key):
        if isinstance(key, int):
            return key
        h = 0
        for ch in str(key):                 # deterministic string hash
            h = h * 31 + ord(ch)
        return h

    def _index(self, key):
        return self._hash(key) & ((1 << self.global_depth) - 1)   # last 'gd' bits

    def insert(self, key, value):
        bucket = self.directory[self._index(key)]
        if key in bucket.items or len(bucket.items) < bucket.capacity:
            bucket.items[key] = value
            return
        if self.global_depth >= 16:
            raise RuntimeError("Too many colliding keys (max depth reached).")
        if bucket.depth == self.global_depth:   # directory must double first
            self.directory = self.directory + self.directory
            self.global_depth += 1
        self._split(bucket)
        self.insert(key, value)                 # retry after split

    def _split(self, bucket):
        bucket.depth += 1
        new_bucket = Bucket(bucket.depth, self.capacity)
        bit = 1 << (bucket.depth - 1)
        for i in range(len(self.directory)):    # re-point half the directory entries
            if self.directory[i] is bucket and (i & bit):
                self.directory[i] = new_bucket
        old_items, bucket.items = bucket.items, {}
        for k, v in old_items.items():          # redistribute keys
            target = new_bucket if self._hash(k) & bit else bucket
            target.items[k] = v

    def search(self, key):
        return self.directory[self._index(key)].items.get(key)
    def delete(self, key):
```

```
        bucket = self.directory[self._index(key)]
        if key in bucket.items:
            del bucket.items[key]
            print(f"Key {key} deleted.")
        else:
            print("Key not found.")

    def display(self):
        print(f"Global depth = {self.global_depth}")
        seen = []
        for i, b in enumerate(self.directory):
            tag = "" if any(b is s for s in seen) else f"  items={b.items}"
            if not tag:
                tag = "  (same bucket as above)"
            else:
                seen.append(b)
            print(f"  dir[{i:0{self.global_depth}b}] -> bucket (local depth {b.depth}){tag}")


if __name__ == "__main__":
    ht = ExtendibleHashTable(bucket_capacity=2)
    while True:
        print("\n1.Insert  2.Search  3.Delete  4.Display  5.Exit")
        ch = input("Choice: ").strip()
        if ch == "1":
            raw = input("Key (integer or text): ")
            key = int(raw) if raw.lstrip("-").isdigit() else raw
            ht.insert(key, input("Value: "))
        elif ch == "2":
            raw = input("Key to search: ")
            key = int(raw) if raw.lstrip("-").isdigit() else raw
            v = ht.search(key)
            print(f"Value = {v}" if v is not None else "Key not found.")
        elif ch == "3":
            raw = input("Key to delete: ")
            ht.delete(int(raw) if raw.lstrip("-").isdigit() else raw)
        elif ch == "4":
            ht.display()
        elif ch == "5":
            break
        else:
            print("Invalid choice.")
# Search O(1); insert O(1) amortised (a split costs O(directory size)).
```

Group D: Graphs and Trees

Question 1

Consider a particular area in your city. Note the popular locations A, B, C . . . in that area. Assume these locations represent nodes of a graph. If there is a route between two locations, it is represented as connections between nodes. Find out the sequence in which you will visit these locations, starting from (say A) using (i) BFS and (ii) DFS. Represent a given graph using an adjacency matrix to perform DFS and an adjacency list to perform BFS.

Python code (D1.py)

```
# Visiting popular locations in an area: BFS (adjacency list) and DFS (adjacency matrix)
from collections import deque

locations = ["A", "B", "C", "D", "E", "F", "G"]          # e.g. A=Station, B=Market ...
routes = [("A", "B"), ("A", "C"), ("B", "D"), ("B", "E"),
          ("C", "F"), ("E", "G"), ("F", "G")]            # undirected routes
idx = {name: i for i, name in enumerate(locations)}
n = len(locations)

# ---- Adjacency list (used for BFS) ----
adj_list = {v: [] for v in locations}
for u, v in routes:
    adj_list[u].append(v)
    adj_list[v].append(u)
for v in adj_list:
    adj_list[v].sort()

# ---- Adjacency matrix (used for DFS) ----
adj_matrix = [[0] * n for _ in range(n)]
for u, v in routes:
    adj_matrix[idx[u]][idx[v]] = adj_matrix[idx[v]][idx[u]] = 1


def bfs(start):
    visited, order = {start}, []
    queue = deque([start])
    while queue:
        node = queue.popleft()
        order.append(node)
        for nb in adj_list[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append(nb)
    return order                                    # O(V + E)


def dfs(start):
    visited, order = [False] * n, []

    def visit(u):
        visited[u] = True
        order.append(locations[u])
        for v in range(n):                          # scan a whole matrix row: O(V)
            if adj_matrix[u][v] == 1 and not visited[v]:
                visit(v)

    visit(idx[start])
    return order                                    # O(V^2)


if __name__ == "__main__":
    print("Adjacency list:")
    for v, nbrs in adj_list.items():
        print(f"  {v}: {nbrs}")
    print("\nAdjacency matrix:")
    print("    " + "  ".join(locations))
    for i, row in enumerate(adj_matrix):
        print(f"{locations[i]} | " + "  ".join(map(str, row)))

    start = input("\nStart location (e.g. A): ").strip().upper() or "A"
    print("BFS order:", " -> ".join(bfs(start)))
    print("DFS order:", " -> ".join(dfs(start)))
```

Question 2

A pizza shop receives multiple orders from several locations. Assume that one pizza boy is tasked with delivering pizzas in nearby locations, which is represented using a graph. The time required to reach from one location to another represents node connections. Solve the problem of delivering a pizza to all customers in the minimum time. Use appropriate data structures.

Python code (D2.py)

```
# Pizza delivery in minimum time
# Graph: nodes = shop + customer locations, edge weight = travel time (minutes).
# Step 1: Dijkstra (min-heap)  -> shortest travel time between every pair of locations.
# Step 2: Held-Karp bitmask DP -> best order to visit ALL customers with minimum total time.
import heapq

graph = {                       # undirected weighted graph (time in minutes)
    "Shop": {"A": 4, "B": 2},
    "A":    {"Shop": 4, "B": 1, "C": 5},
    "B":    {"Shop": 2, "A": 1, "C": 8, "D": 10},
    "C":    {"A": 5, "B": 8, "D": 3},
    "D":    {"B": 10, "C": 3},
}
INF = float("inf")


def dijkstra(src):
    dist = {v: INF for v in graph}
    dist[src] = 0
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u].items():
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return dist                                     # O((V + E) log V)


def best_delivery_route(start="Shop", return_to_shop=True):
    customers = [v for v in graph if v != start]
    k = len(customers)
    dist = {v: dijkstra(v) for v in graph}          # all-pairs shortest times

    # dp[mask][i] = least time to start at 'start', visit customers in mask, end at customer i
    dp = [[INF] * k for _ in range(1 << k)]
    parent = [[-1] * k for _ in range(1 << k)]
    for i in range(k):
        dp[1 << i][i] = dist[start][customers[i]]
    for mask in range(1 << k):
        for i in range(k):
            if dp[mask][i] == INF or not (mask >> i) & 1:
                continue
            for j in range(k):
                if (mask >> j) & 1:
                    continue
                nmask = mask | (1 << j)
                cost = dp[mask][i] + dist[customers[i]][customers[j]]
                if cost < dp[nmask][j]:
                    dp[nmask][j] = cost
                    parent[nmask][j] = i

    full = (1 << k) - 1
    best, last = INF, -1
    for i in range(k):
        total = dp[full][i] + (dist[customers[i]][start] if return_to_shop else 0)
        if total < best:
            best, last = total, i

    order, mask = [], full                          # rebuild the route
    while last != -1:
        order.append(customers[last])
        prev = parent[mask][last]
        mask ^= 1 << last
```

```
        last = prev
    order.reverse()
    return best, [start] + order + ([start] if return_to_shop else [])


if __name__ == "__main__":
    print("Shortest time from Shop to each location:")
    for place, t in dijkstra("Shop").items():
        print(f"  {place}: {t} min")

    total, route = best_delivery_route("Shop", return_to_shop=True)
    print("\nBest delivery order (each leg follows the shortest path):\n ", " -> ".join(route))
    print("Total time (minutes):", total)
# Dijkstra: O((V+E) log V) per source.  Held-Karp: O(2^k * k^2) for k customers
# (the exact problem is the Travelling Salesman Problem, so it is only practical for small k).
```

Question 3

Implement various operations on a Binary Search Tree, such as insertion, deletion, display, and search.

Python code (D3.py)

```
# Binary Search Tree: insertion, deletion, display, search
class Node:
    def __init__(self, key):
        self.key = key
        self.left = self.right = None


class BST:
    def __init__(self):
        self.root = None

    def insert(self, root, key):
        if root is None:
            return Node(key)
        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            print("Duplicate key ignored.")
        return root

    def search(self, root, key):
        while root and root.key != key:
            root = root.left if key < root.key else root.right
        return root is not None

    def _min_node(self, node):
        while node.left:
            node = node.left
        return node

    def delete(self, root, key):
        if root is None:
            print("Key not found.")
            return None
        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:                # 0 or 1 child
                return root.right
            if root.right is None:
                return root.left
            succ = self._min_node(root.right)    # 2 children: inorder successor
            root.key = succ.key
            root.right = self.delete(root.right, succ.key)
        return root

    def inorder(self, r, out):
        if r: self.inorder(r.left, out); out.append(r.key); self.inorder(r.right, out)
        return out

    def preorder(self, r, out):
        if r: out.append(r.key); self.preorder(r.left, out); self.preorder(r.right, out)
```

```
        return out

    def postorder(self, r, out):
        if r: self.postorder(r.left, out); self.postorder(r.right, out); out.append(r.key)
        return out


if __name__ == "__main__":
    t = BST()
    while True:
        print("\n1.Insert  2.Delete  3.Display  4.Search  5.Exit")
        ch = input("Choice: ").strip()
        if ch == "1":
            t.root = t.insert(t.root, int(input("Key: ")))
        elif ch == "2":
            t.root = t.delete(t.root, int(input("Key to delete: ")))
        elif ch == "3":
            print("Inorder  :", t.inorder(t.root, []))
            print("Preorder :", t.preorder(t.root, []))
            print("Postorder:", t.postorder(t.root, []))
        elif ch == "4":
            print("Found" if t.search(t.root, int(input("Key to search: "))) else "Not found")
        elif ch == "5":
            break
        else:
            print("Invalid choice.")
# Insert/search/delete: O(h) -> O(log n) average, O(n) worst (skewed tree). Traversals O(n).
```

Question 4

Construct an expression tree from the given prefix expression, e.g., +--a*bc/def, traverse it using post-order traversal (non-recursive), and then delete the entire tree.

Python code (D4.py)

```
# Expression tree from a PREFIX expression, non-recursive post-order traversal, tree deletion
class Node:
    def __init__(self, data):
        self.data = data
        self.left = self.right = None


def build_from_prefix(expr):
    """Scan right to left; operands are pushed, an operator pops two subtrees."""
    stack = []
    for ch in reversed(expr.replace(" ", "")):
        node = Node(ch)
        if ch in "+-*/^":
            node.left = stack.pop()       # first operand
            node.right = stack.pop()      # second operand
        stack.append(node)
    return stack.pop()


def postorder_iterative(root):
    """Left -> Right -> Root using one stack and a 'last visited' pointer."""
    result, stack, last, cur = [], [], None, root
    while cur or stack:
        if cur:
            stack.append(cur)
            cur = cur.left
        else:
            peek = stack[-1]
            if peek.right and last is not peek.right:
                cur = peek.right
            else:
                result.append(peek.data)
                last = stack.pop()
    return result


def delete_tree(root):
    """Delete nodes bottom-up (post-order) without recursion; returns count deleted."""
    if not root:
        return 0
    stack, count = [root], 0
    while stack:
```

```
        node = stack.pop()
        if node.left:  stack.append(node.left)
        if node.right: stack.append(node.right)
        print(f"Deleting node: {node.data}")
        node.left = node.right = None          # break links
        count += 1
    return count


if __name__ == "__main__":
    prefix = input("Enter prefix expression (default +--a*bc/def): ").strip() or "+--a*bc/def"
    root = build_from_prefix(prefix)
    print("Post-order :", " ".join(postorder_iterative(root)))
    print("Nodes deleted:", delete_tree(root))
    root = None
# All steps are O(n) time; stack space O(h) (O(n) worst case).
```

Question 5

A list stores city names and their populations. Use a Binary Search Tree for implementation. Provide a facility for adding new cities, deleting a city, and updating population values. Provide a facility to display all the city names in ascending/descending order. Also, find how many maximum comparisons are required to search for a particular city.

Python code (D5.py)

```
# Cities and populations stored in a Binary Search Tree (key = city name)
class CityNode:
    def __init__(self, name, population):
        self.name, self.population = name, population
        self.left = self.right = None


class CityBST:
    def __init__(self):
        self.root = None

    def add(self, name, population):
        node = CityNode(name, population)
        if not self.root:
            self.root = node
            return
        cur = self.root
        while True:
            if name.lower() == cur.name.lower():
                print("City already exists. Use update.")
                return
            side = "left" if name.lower() < cur.name.lower() else "right"
            nxt = getattr(cur, side)
            if nxt is None:
                setattr(cur, side, node)
                return
            cur = nxt

    def update(self, name, population):
        cur = self.root
        while cur and cur.name.lower() != name.lower():
            cur = cur.left if name.lower() < cur.name.lower() else cur.right
        if cur:
            cur.population = population
            print("Population updated.")
        else:
            print("City not found.")

    def delete(self, root, name):
        if root is None:
            print("City not found.")
            return None
        if name.lower() < root.name.lower():
            root.left = self.delete(root.left, name)
        elif name.lower() > root.name.lower():
            root.right = self.delete(root.right, name)
        else:
            if root.left is None:
                return root.right
            if root.right is None:
```

```
                return root.left
            succ = root.right
            while succ.left:
                succ = succ.left
            root.name, root.population = succ.name, succ.population
            root.right = self.delete(root.right, succ.name)
        return root

    def display(self, descending=False):
        out, stack, cur = [], [], self.root
        while cur or stack:                      # iterative inorder (or reverse inorder)
            if cur:
                stack.append(cur)
                cur = cur.right if descending else cur.left
            else:
                cur = stack.pop()
                out.append((cur.name, cur.population))
                cur = cur.left if descending else cur.right
        return out

    def comparisons_to_search(self, name):
        count, cur = 0, self.root
        while cur:
            count += 1
            if name.lower() == cur.name.lower():
                return count, True
            cur = cur.left if name.lower() < cur.name.lower() else cur.right
        return count, False

    def height(self, node):
        return 0 if node is None else 1 + max(self.height(node.left), self.height(node.right))


if __name__ == "__main__":
    t = CityBST()
    while True:
        print("\n1.Add City  2.Delete City  3.Update Population  4.Display Ascending")
        print("5.Display Descending  6.Comparisons to search a city  7.Max comparisons  8.Exit")
        ch = input("Choice: ").strip()
        if ch == "1":
            t.add(input("City: "), int(input("Population: ")))
        elif ch == "2":
            t.root = t.delete(t.root, input("City to delete: "))
        elif ch == "3":
            t.update(input("City: "), int(input("New population: ")))
        elif ch in ("4", "5"):
            for name, pop in t.display(descending=(ch == "5")):
                print(f"  {name:<15}{pop:>12,}")
        elif ch == "6":
            c, found = t.comparisons_to_search(input("City to search: "))
            print(f"{c} comparison(s); city {'found' if found else 'NOT found'}.")
        elif ch == "7":
            print("Maximum comparisons for any search = height of tree =", t.height(t.root))
        elif ch == "8":
            break
        else:
            print("Invalid choice.")
# Max comparisons = height h of the BST: about log2(n) when balanced, n when skewed.
```

Question 6

Read the formulas in propositional calculus. Write a function that reads such a formula and creates its binary tree representation. What is the complexity of your function?

Python code (D6.py)

```
# Propositional-calculus formula  ->  binary expression tree
# Symbols : variables (letters), ~ (NOT), & (AND), | (OR), > (IMPLIES), = (IFF), ( )
# Precedence (high -> low): ~  &  |  >  =
class Node:
    def __init__(self, data, left=None, right=None):
        self.data, self.left, self.right = data, left, right


class FormulaParser:
    def __init__(self, text):
        self.tokens = [c for c in text if not c.isspace()]
```

```
        self.pos = 0

    def peek(self):
        return self.tokens[self.pos] if self.pos < len(self.tokens) else None

    def eat(self, expected=None):
        tok = self.peek()
        if tok is None or (expected and tok != expected):
            raise ValueError(f"Syntax error at position {self.pos}")
        self.pos += 1
        return tok

    def parse(self):
        root = self.iff()
        if self.peek() is not None:
            raise ValueError(f"Unexpected '{self.peek()}' at position {self.pos}")
        return root

    def _binary(self, op, next_level):
        node = next_level()
        while self.peek() == op:
            self.eat()
            node = Node(op, node, next_level())
        return node

    def iff(self):     return self._binary("=", self.implies)
    def implies(self):                              # right associative
        left = self.disj()
        if self.peek() == ">":
            self.eat()
            return Node(">", left, self.implies())
        return left
    def disj(self):    return self._binary("|", self.conj)
    def conj(self):    return self._binary("&", self.neg)

    def neg(self):
        if self.peek() == "~":
            self.eat()
            return Node("~", self.neg())            # unary: only left child used
        return self.atom()

    def atom(self):
        tok = self.peek()
        if tok == "(":
            self.eat("(")
            node = self.iff()
            self.eat(")")
            return node
        if tok is not None and tok.isalpha():
            return Node(self.eat())
        raise ValueError(f"Syntax error at position {self.pos}")


def inorder(n):
    if n is None: return ""
    if n.data == "~": return "~" + inorder(n.left)
    if n.left is None: return n.data
    return "(" + inorder(n.left) + n.data + inorder(n.right) + ")"

def prefix(n):  return "" if n is None else n.data + prefix(n.left) + prefix(n.right)
def postfix(n): return "" if n is None else postfix(n.left) + postfix(n.right) + n.data

def show(n, level=0):
    if n:
        show(n.right, level + 1)
        print("      " * level + n.data)
        show(n.left, level + 1)


if __name__ == "__main__":
    formula = input("Enter formula (e.g. (p&q)>~r|s): ")
    try:
        root = FormulaParser(formula).parse()
    except ValueError as e:
        print(e)
    else:
        print("Fully parenthesised:", inorder(root))
        print("Prefix  :", prefix(root))
        print("Postfix :", postfix(root))
```

```
        print("Tree (rotated 90 degrees, root at left):")
        show(root)
# Complexity: every token is consumed exactly once -> Time O(n), Space O(n) for the tree
# (plus O(depth) recursion stack, at most O(n)).
```